CHANDIGARH UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

ON

SORTING VISUALIZER

An Interactive Demonstration of Sorting Algorithms

Submitted by	Name: Ziya Iqbal , Himjit Kaur , Mitali Singh
UID	25BCS12240, 25BCS12311, 25BCS11256
Class / Section	25BCS802-B
Subject	Advanced Data Structures (ADS)
Submitted to	Faculty Name: MD Tauseef

CERTIFICATE

This is to certify that the project report entitled “Sorting Visualizer: An Interactive Demonstration of Sorting Algorithms” is submitted by the student whose details are mentioned on the cover page, in partial fulfilment of the academic requirements of the subject Advanced Data Structures (ADS).

The project demonstrates the working of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort through a browser-based visual interface. The work includes the design of the user interface, implementation of algorithm logic using JavaScript, animation of array elements, and basic testing.

The work has been completed under the guidance and supervision of the undersigned.

Faculty Signature:

Name of Faculty: MD Tauseef

Date: 12 NOV 2026

DECLARATION

I hereby declare that this project report titled “Sorting Visualizer” has been prepared by me for the subject Advanced Data Structures (ADS). The project is intended for academic learning and demonstrates the step-by-step operation of commonly used sorting algorithms.

I have made efforts to understand the purpose of each file, the role of HTML and CSS in creating the interface, and the role of JavaScript in implementing and animating the sorting algorithms. Wherever external learning resources or documentation were consulted, they are acknowledged in the references section.

Student Name: Ziya Iqbal, Himjit Kaur, Mitali Singh

UID: 25BCS12240, 25BCS12311, 25BCS11256

Section: 25BCS802-B

Signature:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Chandigarh University for providing us with the opportunity, facilities and resources to successfully complete our project titled “Sorting Visualizer: An Interactive Demonstration of Sorting Algorithms.” We are especially thankful to our project guide, Mr. MD Tauseef, for his valuable guidance, continuous encouragement and constructive suggestions throughout the development of this project. His support helped us understand the concepts of sorting algorithms and implement them effectively.

We would also like to thank the Department of Computer Science and Engineering and all our faculty members for providing us with the knowledge and academic support required to complete this work. We are equally grateful to our teammates for their cooperation, dedication and combined efforts in planning, designing, coding and testing the application. Their contributions helped us overcome challenges and improve the overall functionality of the project.

This project has provided us with an opportunity to apply our theoretical knowledge of Advanced Data Structures and Algorithms in a practical manner. It has enhanced our understanding of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort, along with their implementation and time complexity. Finally, we extend our heartfelt thanks to our families and friends for their constant motivation and support throughout the completion of this project.

ABSTRACT

Sorting is one of the most fundamental operations in computer science. It involves arranging a collection of data elements in a specific order, generally ascending or descending. Sorting plays an important role in various applications, including data organization, searching, database management and information processing. Different sorting algorithms use different techniques to arrange data, and each algorithm has its own working procedure, time complexity and memory requirements. Understanding these algorithms theoretically can sometimes be difficult for students because the actual process of comparisons, swapping and movement of elements is not always easy to visualize.

The Sorting Visualizer is an interactive, browser-based educational application developed to demonstrate the working of five commonly used sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort. The application represents an array of numerical values using individual numbered cards and displays the sorting process through animations. Users can select their preferred sorting algorithm and observe how elements are compared, exchanged, shifted and arranged into their correct positions.

The application provides several interactive features, including random array generation, algorithm selection, start, pause, resume and reset controls, along with an animation speed slider. Different colours are used to highlight unsorted elements, comparisons, swapping operations and sorted positions. It also displays comparison and swap or movement counters, helping users observe the operations performed during execution.

The project is developed using HTML, CSS and JavaScript. HTML provides the structure of the application, CSS manages the interface design and animation effects, while JavaScript implements the sorting algorithms and handles user interactions. The application runs directly in a modern web browser without requiring a database or backend server.

The main aim of this project is to make sorting algorithms easier to understand through visual representation and practical demonstration. It helps students connect theoretical concepts of Data Structures and Algorithms with their actual implementation, while improving their understanding of algorithm execution, comparisons, recursion and sorting techniques.

TABLE OF CONTENTS

Section	Page
1. Introduction	7
2. Problem Statement and Objectives	9
3. System Requirements and Technology	9
4. System Design and Working	10
5. Sorting Algorithms	11
6. Implementation and User Interface	12-13
7. Source Code – HTML and CSS	14-17
8. Source Code – JavaScript	18-22
9. Testing, Results and Limitations	23
10. Conclusion and Future Scope	24
References	24

1. INTRODUCTION

1.1 Background

Sorting means arranging a collection of elements according to a chosen order. In ascending order, smaller values appear before larger values. In descending order, the arrangement is reversed. Sorting is important because organized data is easier to search, read, process and analyze.

Different sorting algorithms solve the same basic problem using different strategies. Some repeatedly compare adjacent values, some select the smallest remaining value, while others divide the array into smaller parts. Their execution time and memory requirements can vary significantly depending on the input size and initial arrangement.

1.2 Need for Visualization

When a student studies an algorithm only through written steps, it may be difficult to imagine how the array changes after each operation. A visualizer makes these operations visible. In this project, each number is displayed inside a card. A highlighted card indicates a comparison, a red card indicates a movement or exchange, and a green card indicates a position that has been finalized.

1.3 Project Overview

The project runs locally in a modern web browser and does not require a database or server. The user opens index.html, selects an algorithm and starts the animation. JavaScript updates the array and redraws the cards after each operation. CSS transitions make the visual changes smoother.

1.4 Scope

- Educational demonstration of five common sorting algorithms.
- Visual representation using numbered cards instead of a conventional bar graph.
- User controls for input randomization, speed, pause, resume and reset.
- Basic counters for comparisons and swaps or moves.
- Local execution using a browser without an internet connection.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Students often understand the definition of a sorting algorithm but find it challenging to follow the exact sequence of comparisons and movements. Static examples show only the input and final output, not the intermediate operations. The problem addressed by this project is to provide a simple interactive application that displays these intermediate steps in an understandable manner.

2.2 Objectives

- To create a simple and user-friendly interface for learning sorting algorithms.
- To represent array elements as numbered cards arranged horizontally.
- To highlight elements being compared and moved during execution.
- To implement Bubble, Selection, Insertion, Merge and Quick Sort.
- To provide controls for starting, pausing, resuming and resetting an animation.
- To allow users to generate a new random array and change animation speed.
- To display comparison and swap/move counts for basic observation.

2.3 Expected Outcome

The expected outcome is a working browser-based visualizer in which the user can select an algorithm and see the array gradually become sorted. At completion, all cards are displayed in ascending order and marked with the sorted colour. The project should help learners connect algorithm logic with the changes occurring in an array.

2.4 Intended Users

The application is mainly intended for undergraduate computer science students, beginners learning data structures, and anyone who wants to understand the basic working of sorting algorithms.

2.5 GitHub Repository Link:

https://github.com/ZiyaIqbal/Sorting-Visualizer

3. SYSTEM REQUIREMENTS AND TECHNOLOGY

3.1 Hardware Requirements

- A desktop computer or laptop.
- At least 2 GB RAM recommended for normal browser use.
- Keyboard, mouse or touchpad.

3.2 Software Requirements

- Operating system: Windows, Linux or macOS.
- A modern browser such as Google Chrome, Microsoft Edge or Firefox.
- A code editor such as Visual Studio Code or any text editor.
- No database or external package is required for the basic version.

3.3 Technologies Used

Technology	**Purpose**	**Project file / use**
HTML	Defines page structure and controls	index.html
CSS	Styles cards, buttons, colours and transitions	style.css
JavaScript	Implements algorithms and event handling	script.js
Web browser	Runs the application	Opens index.html

3.4 Why These Technologies?

HTML, CSS and JavaScript are suitable because they are supported directly by modern browsers. HTML creates the controls and visualizer container, CSS provides layout and animation effects, and JavaScript changes the values and classes dynamically. This combination keeps the project lightweight and easy to run.

4. SYSTEM DESIGN AND WORKING

4.1 Main Modules

- Input generation module: creates a random list of numbers and stores the original array.
- Algorithm selection module: reads the selected sorting method from the dropdown.
- Sorting module: performs comparisons and updates the array according to the selected algorithm.
- Rendering module: creates numbered card elements and applies visual classes.
- Control module: manages start, pause, resume, reset and randomize actions.
- Statistics module: updates comparison and movement counters.

4.2 Working Flow

1. The user opens the index.html file in a browser.
2. The application generates a random array and displays it as numbered cards.
3. The user chooses one of the five sorting algorithms.
4. The Start Sorting button calls the selected algorithm.
5. During execution, JavaScript compares values and updates the array.
6. The rendering function redraws the cards with comparison or movement highlights.
7. After completion, all cards are marked as sorted.

4.3 Colour Representation

Colour	**Meaning**
Blue	Unsorted element
Yellow	Element currently being compared
Red	Element being moved or exchanged
Green	Element or position marked as sorted

4.4 Animation Control

The delay function uses JavaScript promises and timers to create a pause between operations. The speed slider changes the delay. The pause control temporarily waits before the next operation, while reset stops the current run and restores the original array.

5. SORTING ALGORITHMS

5.1 Bubble Sort

Bubble Sort repeatedly compares adjacent elements. If the left element is greater than the right element, the two values are exchanged. After each complete pass, the largest remaining value reaches its correct position at the end of the unsorted portion.

Time complexity: best case O(n) with an optimized implementation; average and worst case $O(n^2)$. Space complexity: O(1). The current project uses the straightforward version, so it continues the passes even when no swap occurs.

5.2 Selection Sort

Selection Sort divides the array into a sorted part and an unsorted part. It finds the smallest value in the unsorted portion and exchanges it with the first unsorted position. This process continues until all positions are finalized.

Time complexity: best, average and worst case $O(n^2)$. Space complexity: O(1). It performs relatively few exchanges compared with some simple sorting methods.

5.3 Insertion Sort

Insertion Sort builds a sorted portion from left to right. It takes the next value as a key and shifts larger values one position to the right until the correct place for the key is found.

Time complexity: best case O(n), average and worst case $O(n^2)$. Space complexity: O(1). It is useful for small or nearly sorted arrays.

5.4 Merge Sort

Merge Sort follows the divide-and-conquer approach. It divides the array into two halves, recursively sorts each half and then merges the sorted halves into a single sorted sequence.

Time complexity: O(n log n) in best, average and worst cases. The typical auxiliary space requirement is O(n).

5.5 Quick Sort

Quick Sort selects a pivot value and partitions the array so that values smaller than the pivot are placed before it and larger values are placed after it. The two partitions are then sorted recursively.

Average time complexity: O(n log n). Worst case: $O(n^2)$, depending on pivot selection and input arrangement. The in-place partitioning approach used here requires recursion stack space.

6. IMPLEMENTATION AND USER INTERFACE

6.1 Project Folder Structure

The project contains three main files. All files should be saved in the same folder so that the browser can correctly load the stylesheet and JavaScript file.

```
Sorting-Visualizer/
|-- index.html
|-- style.css
|-- script.js
```

6.2 HTML Implementation

The HTML file contains the page heading, algorithm dropdown, control buttons, speed slider, visualizer area, colour legend and statistics. It also links the CSS file in the head section and the JavaScript file before the closing body tag.

6.3 CSS Implementation

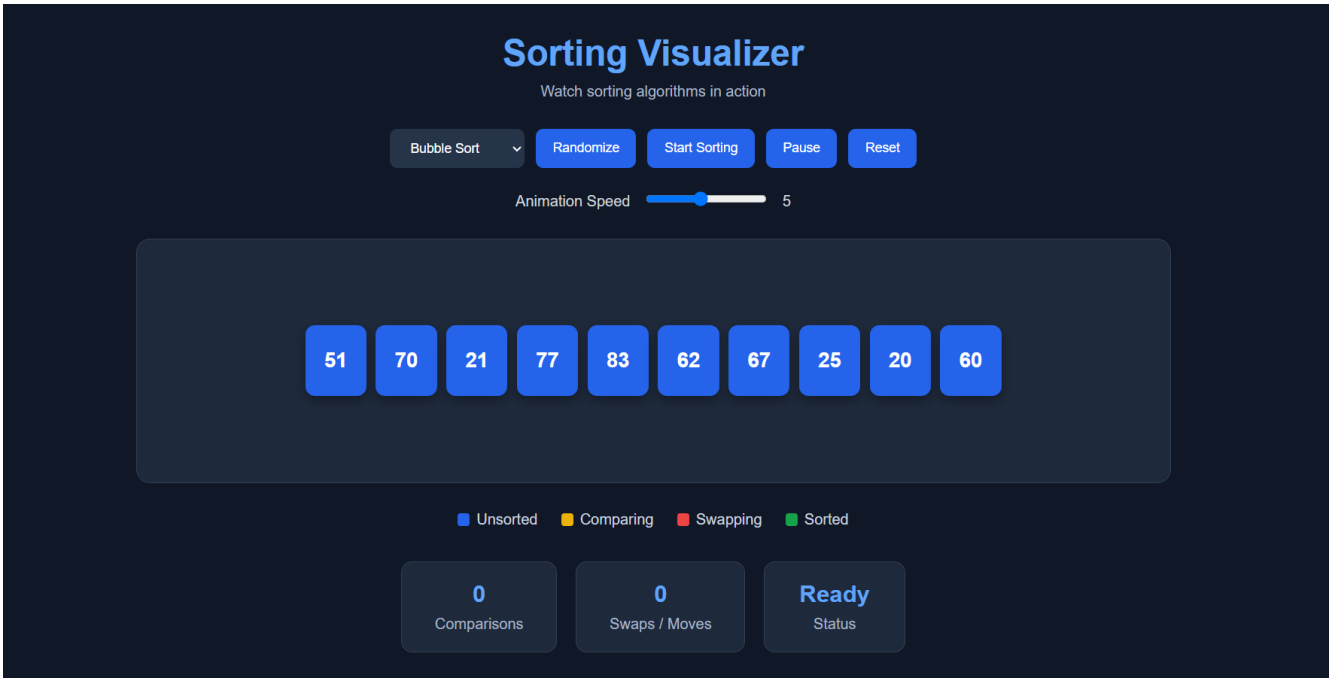
The CSS file controls the dark background, card dimensions, spacing, button appearance and responsive layout. The transition property gives the cards a smooth visual change when their state or position is updated. The comparison and swap classes use different colours to make the current operation easy to identify.

6.4 JavaScript Implementation

The JavaScript file generates the input array, renders the cards and implements each sorting method. Event listeners connect the buttons and slider to their actions. Asynchronous functions and delay handling allow the user to observe the intermediate steps.

6.5 Frontend Screenshots – Insert Your Own Images

Paste screenshots of your running application in the spaces below. Use screenshots captured from your own browser after running the project.




Sorting Visualizer
Watch sorting algorithms in action
Insertion Sort
Randomize
Start Sorting
Resume
Reset
Animation Speed
10
42
61
76
82
89
89
96
96
38
55
Unsorted
Comparing
Swapping
Sorted
12
Comparisons
6
Swaps / Moves
Paused
Status


Sorting Visualizer
Watch sorting algorithms in action
Merge Sort
Randomize
Start Sorting
Pause
Reset
Animation Speed
10
20
21
25
51
60
62
67
70
77
83
Unsorted
Comparing
Swapping
Sorted
19
Comparisons
34
Swaps / Moves
Completed!
Status

7. SOURCE CODE – HTML AND CSS

7.1 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sorting Visualizer</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Sorting Visualizer</h1>
        <p class="subtitle">Watch sorting algorithms in action</p>
        <div class="controls">
            <select id="algorithm">
                <option value="bubble">Bubble Sort</option>
                <option value="selection">Selection Sort</option>
                <option value="insertion">Insertion Sort</option>
                <option value="merge">Merge Sort</option>
                <option value="quick">Quick Sort</option>
            </select>
            <button id="randomBtn">Randomize</button>
            <button id="startBtn">Start Sorting</button>
            <button id="pauseBtn">Pause</button>
            <button id="resetBtn">Reset</button>
        </div>
        <div class="speed-control">
            <label>Animation Speed</label>
            <input type="range" id="speed" min="1" max="10" value="5">
            <span id="speedValue">5</span>
        </div>
        <div class="visualizer" id="visualizer"></div>
        <div class="legend">
            <span><i class="blue"></i> Unsorted</span>
            <span><i class="yellow"></i> Comparing</span>
            <span><i class="red"></i> Swapping</span>
            <span><i class="green"></i> Sorted</span>
        </div>
        <div class="statistics">
            <div class="stat"><h3 id="comparisons">0</h3><p>Comparisons</p></div>
            <div class="stat"><h3 id="swaps">0</h3><p>Swaps / Moves</p></div>
            <div class="stat"><h3 id="status">Ready</h3><p>Status</p></div>
        </div>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

7. SOURCE CODE – HTML AND CSS (CONTINUED)

7.2 style.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
body {
    font-family: Arial, sans-serif;
    background: #101827;
    color: white;
    min-height: 100vh;
    padding: 35px 20px;
}
.container {
    max-width: 1100px;
    margin: auto;
    text-align: center;
}
h1 {
    font-size: 38px;
    color: #60a5fa;
    margin-bottom: 10px;
}
.subtitle {
    color: #aab7ca;
    margin-bottom: 30px;
}
.controls {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    gap: 12px;
    margin-bottom: 25px;
}
button, select {
    padding: 12px 18px;
    border: none;
    border-radius: 8px;
    font-size: 14px;
    cursor: pointer;
}
button {
    background: #2563eb;
    color: white;
}
button:hover { background: #1d4ed8; }
button:disabled {
    opacity: 0.45;
    cursor: not-allowed;
}
select {
    background: #263449;
    color: white;
}
.speed-control {
    margin: 20px;
    color: #d1d5db;
}
.speed-control input {
    margin: 0 12px;
    cursor: pointer;
```

```
}
.visualizer {
    background: #1e293b;
    border: 1px solid #334155;
    border-radius: 15px;
    min-height: 260px;
    padding: 30px 20px;
    display: flex;
    justify-content: center;
    align-items: center;
    flex-wrap: wrap;
    gap: 10px;
    margin: 30px 0;
}
.card {
    width: 65px;
    height: 75px;
    background: #2563eb;
    border-radius: 10px;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 22px;
    font-weight: bold;
    transition: transform 0.35s ease, background 0.25s ease;
    box-shadow: 0 5px 12px #0005;
}
.card.comparing {
    background: #eab308;
    color: #111827;
    transform: translateY(-15px);
}
.card.swapping {
    background: #ef4444;
    transform: translateY(15px);
}
.card.sorted { background: #16a34a; }
.legend {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    gap: 25px;
    color: #cbd5e1;
    margin-bottom: 35px;
}
.legend span {
    display: flex;
    align-items: center;
    gap: 7px;
}
.legend i {
    width: 13px;
    height: 13px;
    display: inline-block;
    border-radius: 3px;
}
.blue { background: #2563eb; }
.yellow { background: #eab308; }
.red { background: #ef4444; }
.green { background: #16a34a; }
.statistics {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
```

```
    gap: 20px;
}
.stat {
    background: #1e293b;
    border: 1px solid #334155;
    border-radius: 12px;
    padding: 20px 35px;
    min-width: 150px;
}
.stat h3 {
    color: #60a5fa;
    font-size: 25px;
    margin-bottom: 8px;
}
.stat p { color: #aab7ca; }
@media (max-width: 600px) {
    .card {
        width: 45px;
        height: 55px;
        font-size: 17px;
    }
    h1 { font-size: 29px; }
}
```

8. SOURCE CODE – JAVASCRIPT

8.1 script.js – Array generation, rendering, delay handling and Bubble Sort

```
const visualizer = document.getElementById("visualizer");
const algorithm = document.getElementById("algorithm");
const randomBtn = document.getElementById("randomBtn");
const startBtn = document.getElementById("startBtn");
const pauseBtn = document.getElementById("pauseBtn");
const resetBtn = document.getElementById("resetBtn");
const speedInput = document.getElementById("speed");
const speedValue = document.getElementById("speedValue");

let original = [];
let arr = [];
let comparisons = 0;
let swaps = 0;
let paused = false;
let stopped = false;
let running = false;
let sortedIndices = new Set();

function generateArray() {
    original = [];
    for (let i = 0; i < 10; i++) {
        original.push(Math.floor(Math.random() * 90) + 10);
    }
    arr = [...original];
    comparisons = 0;
    swaps = 0;
    sortedIndices.clear();
    updateStats();
    render();
    document.getElementById("status").textContent = "Ready";
}

function render(highlight = [], swap = []) {
    visualizer.innerHTML = "";
    arr.forEach((value, index) => {
        const card = document.createElement("div");
        card.className = "card";
        card.textContent = value;
        if (sortedIndices.has(index)) card.classList.add("sorted");
        if (highlight.includes(index)) card.classList.add("comparing");
        if (swap.includes(index)) card.classList.add("swapping");
        visualizer.appendChild(card);
    });
}

function updateStats() {
    document.getElementById("comparisons").textContent = comparisons;
    document.getElementById("swaps").textContent = swaps;
}

function delay() {
    return new Promise(resolve => {
        const check = () => {
            if (stopped) {
                resolve();
                return;
            }
            if (!paused) {
                const time = 1100 - speedInput.value * 100;
                setTimeout(resolve, time);
```

```
            } else {
                setTimeout(check, 100);
            }
        };
        check();
    });
}

async function compare(i, j) {
    if (stopped) return;
    comparisons++;
    updateStats();
    render([i, j]);
    await delay();
}

async function exchange(i, j) {
    if (stopped) return;
    [arr[i], arr[j]] = [arr[j], arr[i]];
    swaps++;
    updateStats();
    render([], [i, j]);
    await delay();
}

async function markSorted(index) {
    sortedIndices.add(index);
    render();
    await delay();
}

// BUBBLE SORT
async function bubbleSort() {
    let n = arr.length;
    for (let i = 0; i < n - 1; i++) {
        for (let j = 0; j < n - i - 1; j++) {
            if (stopped) return;
            await compare(j, j + 1);
            if (arr[j] > arr[j + 1]) {
                await exchange(j, j + 1);
            }
        }
        await markSorted(n - i - 1);
    }
    await markSorted(0);
}
```

8. SOURCE CODE – JAVASCRIPT (CONTINUED)

8.2 script.js – Remaining sorting algorithms and user controls

```
// SELECTION SORT
async function selectionSort() {
    let n = arr.length;
    for (let i = 0; i < n - 1; i++) {
        let min = i;
        for (let j = i + 1; j < n; j++) {
            if (stopped) return;
            await compare(min, j);
            if (arr[j] < arr[min]) min = j;
        }
        if (min !== i) await exchange(i, min);
        await markSorted(i);
    }
    await markSorted(n - 1);
}

// INSERTION SORT
async function insertionSort() {
    let n = arr.length;
    for (let i = 1; i < n; i++) {
        let key = arr[i];
        let j = i - 1;
        while (j >= 0) {
            if (stopped) return;
            await compare(j, j + 1);
            if (arr[j] > key) {
                arr[j + 1] = arr[j];
                swaps++;
                updateStats();
                render([], [j, j + 1]);
                await delay();
                j--;
            } else {
                break;
            }
        }
        arr[j + 1] = key;
        render([], [j + 1]);
        await delay();
    }
    for (let i = 0; i < n; i++) sortedIndices.add(i);
    render();
}

// MERGE SORT
async function mergeSort(left, right) {
    if (stopped || left >= right) return;
    let mid = Math.floor((left + right) / 2);
    await mergeSort(left, mid);
    await mergeSort(mid + 1, right);
    if (stopped) return;
    let temp = [];
    let i = left;
    let j = mid + 1;
    while (i <= mid && j <= right) {
        if (stopped) return;
        await compare(i, j);
        if (arr[i] <= arr[j]) {
            temp.push(arr[i]);
            i++;
```

```
        } else {
            temp.push(arr[j]);
            j++;
        }
    }
    while (i <= mid) temp.push(arr[i++]);
    while (j <= right) temp.push(arr[j++]);
    for (let k = 0; k < temp.length; k++) {
        if (stopped) return;
        arr[left + k] = temp[k];
        swaps++;
        updateStats();
        render([], [left + k]);
        await delay();
    }
}

// QUICK SORT
async function quickSort(low, high) {
    if (stopped) return;
    if (low < high) {
        let pivot = await partition(low, high);
        if (stopped) return;
        await quickSort(low, pivot - 1);
        await quickSort(pivot + 1, high);
    }
}

async function partition(low, high) {
    let pivot = arr[high];
    let i = low - 1;
    for (let j = low; j < high; j++) {
        if (stopped) return low;
        await compare(j, high);
        if (arr[j] < pivot) {
            i++;
            if (i !== j) await exchange(i, j);
        }
    }
    if (i + 1 !== high) await exchange(i + 1, high);
    return i + 1;
}

// START SORTING
async function startSorting() {
    if (running) return;
    running = true;
    paused = false;
    stopped = false;
    comparisons = 0;
    swaps = 0;
    sortedIndices.clear();
    updateStats();

    startBtn.disabled = true;
    randomBtn.disabled = true;
    algorithm.disabled = true;
    document.getElementById("status").textContent = "Sorting...";

    const selected = algorithm.value;
    if (selected === "bubble") await bubbleSort();
    else if (selected === "selection") await selectionSort();
    else if (selected === "insertion") await insertionSort();
    else if (selected === "merge") await mergeSort(0, arr.length - 1);
```

```
    else if (selected === "quick") await quickSort(0, arr.length - 1);

    if (!stopped) {
        sortedIndices.clear();
        for (let i = 0; i < arr.length; i++) sortedIndices.add(i);
        render();
        document.getElementById("status").textContent = "Completed!";
    } else {
        document.getElementById("status").textContent = "Stopped";
    }

    running = false;
    paused = false;
    startBtn.disabled = false;
    randomBtn.disabled = false;
    algorithm.disabled = false;
}

// BUTTON EVENTS
randomBtn.addEventListener("click", () => {
    if (!running) generateArray();
});

startBtn.addEventListener("click", startSorting);

pauseBtn.addEventListener("click", () => {
    if (!running) return;
    paused = !paused;
    pauseBtn.textContent = paused ? "Resume" : "Pause";
    document.getElementById("status").textContent =
        paused ? "Paused" : "Sorting...";
});

resetBtn.addEventListener("click", () => {
    stopped = true;
    paused = false;
    running = false;
    arr = [...original];
    sortedIndices.clear();
    comparisons = 0;
    swaps = 0;
    updateStats();
    render();
    startBtn.disabled = false;
    randomBtn.disabled = false;
    algorithm.disabled = false;
    pauseBtn.textContent = "Pause";
    document.getElementById("status").textContent = "Reset";
});

speedInput.addEventListener("input", () => {
    speedValue.textContent = speedInput.value;
});

// INITIAL DISPLAY
generateArray();
```

9. TESTING, RESULTS AND LIMITATIONS

9.1 Testing Approach

The application should be tested by selecting each algorithm separately and observing whether the final sequence is in ascending order. The controls should also be checked to confirm that the application responds correctly to user actions.

Test case	Expected result	Status to verify
Open index.html	Page and initial cards appear	Pass / Fail
Select each algorithm	Selected algorithm runs	Pass / Fail
Randomize	A new set of values appears	Pass / Fail
Pause and Resume	Animation pauses and continues	Pass / Fail
Reset	Original array is restored	Pass / Fail
Speed slider	Animation delay changes	Pass / Fail
Sorted output	Values appear in ascending order	Pass / Fail

9.2 Sample Expected Output

For example, if the input array is [34, 12, 56, 21, 9], the expected ascending output is [9, 12, 21, 34, 56]. The visualizer should show intermediate comparisons and movements before reaching this final order.

9.3 Observations

- Bubble Sort repeatedly compares neighbouring cards.
- Selection Sort searches for the minimum value in the remaining section.
- Insertion Sort shifts larger values to make room for the current key.
- Merge Sort combines smaller sorted sections.
- Quick Sort places values around a selected pivot.

9.4 Limitations

- The current version uses a small array of ten values to keep the animation readable.
- The visualizer is designed for learning and is not a formal speed benchmark.
- The comparison and swaps/moves counters are educational indicators; the move counter is not identical across all algorithms.
- The current card rendering recreates the elements at each step, so it visually highlights changes but does not provide a physically continuous drag animation.

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The Sorting Visualizer project provides an interactive way to understand five important sorting algorithms. By representing values as numbered cards and highlighting comparisons and movements, it makes the internal working of sorting methods easier to follow than a static explanation. The project also demonstrates the use of HTML, CSS and JavaScript in building a small browser-based application.

During the project, the concepts of arrays, comparisons, swapping, recursion, divide-and-conquer, asynchronous functions, event listeners and DOM manipulation were applied. The work helped connect theoretical knowledge from data structures with practical implementation.

10.2 Future Enhancements

- Add a custom input box so users can enter their own numbers.
- Allow users to choose the number of cards.
- Add step-by-step forward and backward controls.
- Display pseudocode alongside the animation.
- Show time and space complexity for the selected algorithm.
- Improve the animation using CSS transforms or a dedicated animation layer so cards visibly travel between positions.
- Add descending order and additional algorithms such as Heap Sort and Counting Sort.

REFERENCES

- MDN Web Docs, HTML: HyperText Markup Language – https://developer.mozilla.org/en-US/docs/Web/HTML
- MDN Web Docs, CSS – https://developer.mozilla.org/en-US/docs/Web/CSS
- MDN Web Docs, JavaScript – https://developer.mozilla.org/en-US/docs/Web/JavaScript
- GeeksforGeeks, Sorting Algorithms – https://www.geeksforgeeks.org/sorting-algorithms/
- Programiz, Sorting Algorithms – https://www.programiz.com/dsa/sorting-algorithm

Note: The screenshots, student details, faculty details and test outcomes should be completed with the actual project information before submission.